«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 - «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR'S GRADUATION THESIS)
Field of Study
09.03.01 – «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»

Тема / Topic	**Generic Bidirectional Type Elaboration via Free Foil** **Обобщенная двунаправленная элаборация типов с помощью Free Foil**

Работу выполнил / Thesis is executed by	**Геворгян Даниел Геворгович** **Daniel Gevorgyan**	подпись / signature
Руководитель выпускной квалификационной работы / Supervisor of Graduation Thesis	**Кудасов Николай Дмитриевич** **Nikolai Kudasov**	подпись / signature

Иннополис, Innopolis, 2027

Contents

1 Introduction **7**

1.1 Motivation 7

1.2 Existing Solutions 9

1.3 Objectives and Contribution 10

1.4 Scope of the Thesis 11

2 Literature Review **12**

2.1 Type Systems in Programming Languages 13

2.1.1 Simply Typed Lambda Calculus and System F 13

2.1.2 Hindley–Milner Type Inference 14

2.2 Bidirectional Typing 15

2.3 Type Elaboration and Typed Syntax 17

2.4 Abstract Syntax Representations 18

2.5 Scope-Safe Representations of Syntax 19

2.6 Free Foil 20

2.7 Pfenning Recipe 21

2.8 Other Generic Approaches 22

2.9 Predecessor Work 23

2.10 Related Languages 27

2.11 Conclusion 28

3 Methodology **30**

4 Implementation **32**

5 Evaluation and Discussion **33**

6 Conclusion **34**

Bibliography cited **35**

A Extra Stuff **38**

B Even More Extra Stuff **39**

List of Tables

2.1 Predecessor theses on generic type checking 24

3.1 This is the title I want to appear in the List of Tables 30

List of Figures

2.1 Declarative typing rules of the Hindley–Milner system 14
2.2 Bidirectional rules for abstraction, application, and the switch from synthesis to checking 22

Abstract

My abstract starts from here.

Chapter 1

Introduction

1.1 Motivation

Type systems classify programs before execution and rule out whole classes of errors at compile time, and they are a central object of study and implementation in programming language research and teaching. New type systems, language extensions, and experimental languages are regularly proposed, and each is usually accompanied by an implementation used to test and demonstrate it [1], [2]. Getting such a checker working quickly is valuable, because it allows the designer to try new rules and examples early. Bidirectional typing [3] is a well-studied basis for such checkers. It splits the typing judgment into two modes, checking a term against a known type and inferring a type from a term, which keeps the number of required type annotations small, supports features for which full inference is impossible, and makes it clear which rule is responsible for a failure.

Implementing a checker correctly is nevertheless difficult, for two reasons. The first is that languages have binders, and binders make scope bugs easy to introduce. A checker must track variables and their types, avoid variable capture

during substitution, and manage the context when going under a binder. The developers of the Dex compiler report that they repeatedly broke the invariants of the name-handling discipline they used, that this became one of the biggest barriers to implementing new language ideas and onboarding new people, and that compiler passes such as type inference are where such mistakes are most likely [4]. The second reason is repetition. Even when scope safety is expressed in the types of the host language, the same code must be written again for every new scope-safe operation [5], and the same traversal, context handling, and dispatch code is rewritten for every new type checker. Generic implementations are valuable precisely because they allow a new language or type theory to be prototyped without writing a new bidirectional algorithm from scratch [1].

Free Foil [6], [7] addresses the first problem. It generates an efficient, intrinsically scoped abstract syntax with capture-avoiding substitution and α-equivalence from a second-order signature or from a grammar, so that a scoping mistake becomes a compile-time error in Haskell. A line of predecessor theses has built generic type-checking infrastructure on top of it. In particular, Tomilovskaia [8] separates the traversal of the syntax tree and the management of scope from the typing rules of a particular language, so that a language designer supplies one checking and one inference case per syntax construct.

The result of such an engine is, however, limited. Each syntax node is wrapped by operations that either verify a term against a type or return the type of that term. The typing information that was established for the subexpressions along the way is discarded, so later stages of language processing cannot use it without repeating the work. A type checker for a realistic language, such as the proof assistant Rzk [9], instead retains typing information throughout the tree. This thesis studies how a generic bidirectional engine over scope-safe syntax

can expose its result in this form, that is, how it can perform type elaboration: producing a complete typed abstract syntax tree rather than only the type of the root expression.

1.2 Existing Solutions

Someone who wants a working type checker for a new language or type system has several options, each with its own limitations.

Writing the checker by hand. The most direct approach is a checker written for one language, either with a naive representation of syntax or with a scope-safe one. This is flexible, but it repeats the traversal, scope, and context code for every language and every extension, and with a naive representation it exposes the implementer to the scoping bugs described above.

Scope-safe syntax libraries. Free Foil and the techniques it builds on [4], [10] remove the scoping problem but leave the type checker itself to be written. Generic programming over syntaxes with binding [5] goes further and provides generic scope checkers and type checkers in a dependently typed setting, but in a different language and ecosystem.

Generic engines on Free Foil. The predecessor theses are closest to this work. Tomilovskaia [8] builds a generic bidirectional engine and reports that, for a System F checker, it reduces the type-checking logic from 199 to 128 lines compared to a direct Free Foil implementation. Smirnova [11] and Deryabkin [12] do the same for Hindley–Milner inference, and Meslmani [13] extends the bidirectional

engine to modal type theories through an abstract context interface. A summary of this line of work is given in Chapter 2.

These engines establish that separating traversal and scope handling from language-specific rules is practical. What none of them provides is an interface whose output is the elaborated, typed syntax of the program. This is the gap addressed by the present thesis.

1.3 Objectives and Contribution

The goal of this thesis is to design and implement a generic bidirectional type elaboration via Free Foil, in which common type-checking machinery is reused across languages and language-specific typing rules stay separate from the generic implementation. The aim is that a working type checker for a new language or type system can be obtained quickly, by writing only the language-specific rules. It continues the design of Tomilovskaia's engine and develops it into a reusable type-elaboration interface. Specifically, this thesis pursues the following objectives:

1. Generalise the existing bidirectional engine into a reusable one with an interface through which a language author defines the checking and inference rules for each syntax construct.

2. Elaborate programs into a complete typed abstract syntax tree, so that a successful check returns the input program together with the typing information established for its subexpressions.

3. Provide a way for a language to introduce mechanisms that cannot be expressed with the basic checking and inference rules alone.

4. Evaluate the approach on type checkers for several type systems, namely System F and Hindley–Milner inference, built on a single core.

The evaluation is organised around three criteria: reuse of the generic infrastructure across the checkers, production of type-annotated syntax, and the ability to implement language-specific extensions.

1.4 Scope of the Thesis

Some limits of the work are deliberate. The check and infer rules of each language are written by hand, deriving them automatically from declarative rules using the Pfenning recipe is not attempted. The thesis does not provide metatheoretical proofs of soundness or completeness for the generic engine, in contrast to the theory-level work discussed above, and its evidence for correctness comes from concrete language implementations and test suites. The focus is the type-elaboration interface and the production of typed syntax. Error reporting, including precise source locations in error messages, is outside the scope of this thesis and is left for future work.

Chapter 2

Literature Review

This chapter reviews the work that motivates the design of a reusable interface for type elaboration. Sections 2.1 and 2.2 introduce the foundations of type systems and bidirectional typing, building on the exposition of Tomilovskaia [8], and examine Hindley–Milner inference, which places new demands on the typing context. Sections 2.4 and 2.6 discuss scope-safe representations of abstract syntax and the Free Foil framework. Section 2.7 presents the Pfenning recipe as a guide for designing bidirectional rules, while Section 2.8 considers other approaches to generic type checking. Section 2.9 examines the predecessor work most directly related to this thesis, and Section 2.10 discusses related tools and languages.

The central concern of this thesis is not only how to perform checking and inference over scope-safe syntax, but also how to expose their result in a useful form. In particular, type elaboration produces a complete typed abstract syntax tree: the output represents the input program together with the typing information established during checking, rather than merely returning the type of the current expression. This perspective motivates the discussion of typed syntax and the relationship between the present work and its immediate predecessor.

2.1 Type Systems in Programming Languages

A type system classifies the terms of a language by the values they compute, and is usually presented as a set of rules that derive typing judgments of the form $\Gamma \vdash e : \tau$. Read as an algorithm, such rules are a specification from which a checker can be written. The two basic systems used in this thesis, the simply typed λ-calculus and System F, are introduced at length by Tomilovskaia [8, Sec. 2.1], so this section only recalls what the later chapters rely on. Hindley–Milner inference receives a fuller treatment, because it raises a requirement that the existing engine does not meet.

2.1.1 Simply Typed Lambda Calculus and System F

The simply typed λ-calculus (STLC) has variables, abstractions whose parameter carries a type annotation, and applications, over function types alone. Its three typing rules are given by Tomilovskaia [8, Fig. 2.2]. A checker follows them directly: a variable is looked up in the context Γ, an abstraction extends Γ with its parameter, and an application compares the argument with the domain of the function type.

System F adds universal polymorphism to this calculus through type abstraction $\Lambda\alpha.\, e$ and type application $e\,[\tau]$, with the two additional rules shown in [8, Fig. 2.4]. The variant implemented there also includes booleans, natural numbers, conditionals, let, for loops, and type annotations [8, Figs. 2.5 and 2.6]. What matters for this thesis is that terms and types live in one syntax with binders for both term variables and type variables, which makes System F a useful test for any generic treatment of binding.

2.1.2 Hindley–Milner Type Inference

The Hindley–Milner system (HM) is the type discipline behind the core of languages such as ML and Haskell. It restricts System F to prenex polymorphism: type schemes $\sigma ::= \tau \mid \forall\, \alpha.\, \sigma$ quantify only at the top, while monotypes τ contain no quantifiers. In exchange, terms carry no type annotations at all, and the type of every term can be inferred. Figure 2.1 shows the declarative rules. A variable is typed by instantiating its scheme, written $\tau \sqsubseteq \sigma$, and let generalises the type of the bound expression over the type variables that do not occur free in the context, $\mathrm{gen}(\Gamma, \tau_1) = \forall\, \bar{\alpha}.\, \tau_1$ with $\bar{\alpha} = \mathrm{ftv}(\tau_1) \setminus \mathrm{ftv}(\Gamma)$.

$$\frac{x : \sigma \in \Gamma \quad \tau \sqsubseteq \sigma}{\Gamma \vdash x : \tau} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda\, x.\, e : \tau_1 \to \tau_2}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\, e_2 : \tau_2}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \mathrm{gen}(\Gamma, \tau_1) \vdash e_2 : \tau_2}{\Gamma \vdash \mathsf{let}\, x = e_1\, \mathsf{in}\, e_2 : \tau_2}$$

Fig. 2.1. Declarative typing rules of the Hindley–Milner system

The difficulty of HM lies in turning these rules into an algorithm. In the abstraction rule the parameter type τ_1 is not given by the program and has to be guessed. An implementation instead introduces a fresh unification variable for it and, at an application, records the requirement that the type of the function be equal to an arrow from the type of the argument to a further fresh variable. These equalities are solved by unification, either immediately, as in Algorithm W, or

after collecting them as constraints, as in constraint-based formulations such as the level-based variant of Deryabkin [12]. In both cases the checker must maintain, alongside the ordinary typing context Γ, a context of unification constraints, or equivalently the substitution built up so far, which is threaded through the traversal and updated as inference proceeds. Generalisation at let then depends on that state, since the type must first be resolved under the current constraints.

This requirement sets HM apart from the other systems in this section. The rules for the simply typed λ-calculus and System F can be run purely against a scoped environment of variable types, whereas HM inference needs a context that is global to the traversal. Such a constraint context is currently not allowed in Tomilovskaia's engine [8], whose context is a map from the variables in scope to their types [8, Sec. 3.3.1]. The predecessor theses on HM inference [11], [12] therefore provide dedicated implementations of the inference algorithm rather than rules for the bidirectional engine, and supporting HM in a generic interface requires extending the engine so that language-specific rules can read and update such a context.

2.2 Bidirectional Typing

A declarative typing relation $\Gamma \vdash e : \tau$ states what it means for a term to have a type, but not how the type is found. When every binder is annotated, the rules can be read almost directly as a procedure. When annotations are omitted, or when the language includes features such as higher-rank polymorphism or dependent types, for which full inference is undecidable, the relation no longer determines an algorithm. Bidirectional typing addresses this by giving the relation a direction. As surveyed by Dunfield and Krishnaswami [3], it combines two

modes: checking, $\Gamma \vdash e \Leftarrow \tau$, in which the type is supplied from outside and the term is verified against it, and synthesis, $\Gamma \vdash e \Rightarrow \tau$, in which the type is computed from the term itself. Checking makes room for constructs whose types cannot be recovered locally, synthesis spares the programmer from annotating everything, and, because type information is propagated to the construct that needs it, errors are reported closer to their source.

Each rule of a bidirectional system fixes which judgments consume a type and which produce one, and the system is mode-correct when every input of a premise is already available at the point where that premise is used. Two rules connect the modes. An annotation rule turns a synthesis conclusion into a checking premise by supplying the type that the program text states explicitly, and a switch rule, also called subsumption, turns a checking conclusion into a synthesis premise followed by a comparison of the two types. Annotations are consequently needed only where an introduction form, which can only be checked, ends up in a position that requires synthesis, whether directly or through intervening constructs.

The survey assesses a system by four criteria: mode-correctness, as above; completeness, meaning that well-typed programs are not rejected beyond the annotations the system requires; size, since a smaller rule set is easier to maintain; and the character of the annotations, which should be lightweight, predictable, stable under small changes to the program, and legible. It also catalogues variations of the basic discipline, among them local type inference for polymorphism, mixed-direction types, and backwards bidirectional typing. This thesis relies on the basic discipline and the standard rules, and the variations serve mainly as a reminder that the details of a bidirectional system depend on the language it is built for. The presentation of the same discipline in the setting of Free Foil, which this work shares with its predecessor, is given by Tomilovskaia [8, Sec. 2.2].

For a generic implementation, the significance of this discipline is structural. A bidirectional rule says, for one syntactic form, which subterms are checked, which are synthesised, and how their types combine. Everything else a checker does, such as visiting subterms, extending the context under binders, and handling failure, is the same for every form. A rule per construct together with a shared traversal is therefore a natural decomposition, and it leaves room for the traversal to retain the types it establishes at each node, which is taken up next.

2.3 Type Elaboration and Typed Syntax

Type checking is often described in terms of a judgment that relates a term to its type. In an implementation, however, returning the type of the root expression is not always sufficient. Later compiler and language-processing stages may need access to the typing information established for subexpressions as well. A type checker can therefore produce an annotated representation of the program in addition to deciding whether the program is well typed.

In this thesis, *type elaboration* refers to producing a complete typed abstract syntax tree. The result retains the structure of the program while recording the typing information associated with its syntax nodes. This is distinct from an interface that returns only the type inferred for the current node: the annotations are attached throughout the resulting tree, so that subsequent stages can inspect the typing information of subexpressions without repeating the type-checking process.

This distinction is important for the design of a generic engine. Checking and inference remain the two modes through which the typing rules are expressed, but the information computed by those rules contributes to a structured output.

The interface must consequently account not only for the propagation of expected and inferred types, but also for the construction of the typed syntax returned by the checker. The present work investigates this as part of a practical, reusable type-elaboration interface.

2.4 Abstract Syntax Representations

Abstract syntax pairs a signature, which describes the shape of syntax nodes, with a recursive construction that assembles them into trees; the basic notions are summarised in [8, Sec. 2.3]. Swierstra's data types à la carte [14] express syntax as free monads so that language features can be combined incrementally.

Second-order abstract syntax (SOAS) extends signatures with binders, and Kudasov [10] shows how an intrinsically scoped syntax can be generated from a second-order signature using free monads, with the variables in scope tracked in the type of the term. This thesis does not use data types à la carte directly. In the same way as Free Foil, it aims to support that style, so that language extensions can be written as separate signature fragments and combined without rewriting existing code.Making binders explicit in this way raises the question of how scoping is kept correct.

Efficient representations are not immune to such mistakes. The rapier technique used by the Glasgow Haskell Compiler [15] gives fast, stateless capture-avoiding substitution, but it relies on invariants that nothing checks. The developers of the Dex compiler report that they repeatedly broke these invariants, and that this became one of the biggest barriers to implementing new language ideas and to onboarding new people [4]. They also point out that the harder part is not substitution itself but compiler passes such as type inference, which contain logic

similar to substitution and require constant bookkeeping of scopes and renaming, and that this is where mistakes are made.

2.5 Scope-Safe Representations of Syntax

The remedy for unchecked invariants is to have the compiler check them. The foil of Maclaurin, Radul and Paszke [4] does so for the rapier, and its authors report anecdotally that, after they adopted it, the formerly pervasive name-handling bugs disappeared from the Dex compiler.

The same idea can be pursued more generally by letting the host language's type system enforce scope correctness, so that ill-scoped terms cannot be constructed at all. Allais, Atkey, Chapman, McBride and McKinna [5] observe that, without this, programmers must remain constantly vigilant against accidentally constructing ill-scoped terms, because the implicit context is prone to being forgotten or muddled. They also observe that, even when scope safety is expressed in types, the programmer still rewrites the same boilerplate for each new scope-safe operation, such as renaming, substitution, desugaring, or printing, and then again for the correctness proofs. Their remedy is a universe of syntaxes with binding in which scope-safe traversals are written once and for all by generic programming; their examples include a generic scope checker, a bidirectional type checker, and an elaborator into a typed core language. This is the same motivation as the present thesis, applied there to traversals and proofs and here to type checking. The cost of repeated work on the type-checking side is also acknowledged by Felicissimo, who notes that a generic implementation lets one prototype a new type theory without writing a new bidirectional algorithm from scratch, which is valuable given the large number of new theories proposed every year [1].

For this thesis, scope-safe representations matter for a practical reason: typing rules must be implemented over real syntax trees, and they interact non-trivially with binding. A representation that makes scope explicit and machine-checked reduces the risk of capture and shadowing bugs and underpins the generic traversal architecture used later.

2.6 Free Foil

Free Foil [6], [7] implements SOAS as scope-safe abstract syntax in Haskell. Its signatures, the scope-tracking AST type, and the restrictions it places on grammars are described by Tomilovskaia [8, Secs. 2.4 and 3.1], so only the aspects that the rest of this thesis relies on are recalled here.

Free Foil is built on the foil of Maclaurin, Radul and Paszke [4], which enforces the invariants of the rapier technique at compile time using rank-2 polymorphism, generalised algebraic data types, and phantom types. From it Free Foil inherits efficient capture-avoiding substitution and α-equivalence, both of which a type checker needs. Template Haskell functions generate the scope-safe representation from a naive one, so that tools such as the BNF Converter can take a language from a grammar to a parser, a pretty-printer, and an efficient scope-safe syntax with little effort. The benchmarks in the Free Foil paper show it outperforming free scoped monads built on nested de Bruijn indices, while the plain foil is faster still, which the authors attribute to the extra level of indirection introduced for each node.

How Free Foil is applied depends on the binding structure of the language, as discussed in [8, Sec. 2.4]. The case that matters most here is mixed binding, as in System F or dependent type theories, where one syntax covers both terms and

types. The price is a practical constraint on the language designer: the grammar must expose its binding structure by separating plain subterms, scoped subterms, and patterns. This constraint limits how grammars can be written, but it is what makes generic traversal possible.

2.7 Pfenning Recipe

The Pfenning recipe [3] turns a declarative type system into a bidirectional one by assigning modes. The principal judgment of a rule, the one for the connective it introduces or eliminates, is checked for introduction forms and synthesised for elimination forms, and the modes of the remaining premises are then chosen so that every input is known when it is needed. Tomilovskaia [8, Sec. 2.5] gives the steps and works through the application rule, so they are not repeated here. Two further rules mediate between the modes: an annotation rule switches from synthesis in the conclusion to checking in the premise, and a subsumption (switch) rule switches from checking to synthesis, followed by an equality or subtyping test. Annotations are therefore needed where an introduction form meets an elimination form, directly or indirectly.

Figure 2.2 complements that worked example with the rules for abstraction, an introduction form that is checked, and for the switch rule, alongside application.

For a generic checker, the recipe matters because it makes the choice of mode mechanical. In this thesis, however, the recipe serves as a design guide rather than as an algorithm to be automated: the full recipe has more cases than the ones shown here, and automating it is beyond the scope of this work. The check and infer rules of each language are therefore written by hand, as the language-specific part of the engine, while the machinery around them is shared.

$$\frac{\Gamma, x : A \vdash e \Leftarrow B}{\Gamma \vdash \lambda x.\, e \Leftarrow A \to B} \qquad \frac{\Gamma \vdash e \Rightarrow A' \quad A' = A}{\Gamma \vdash e \Leftarrow A}$$

$$\frac{\Gamma \vdash f \Rightarrow A \to B \quad \Gamma \vdash a \Leftarrow A}{\Gamma \vdash f\ a \Rightarrow B}$$

Fig. 2.2. Bidirectional rules for abstraction, application, and the switch from synthesis to checking

2.8 Other Generic Approaches

Genericity in type checkers has been pursued in several complementary ways. The most relevant is the work of Felicissimo [1], who presents a framework for generic bidirectional typing for dependent type theories. The paper observes that the formal development of bidirectional typing had until then stayed confined to specific theories, with general guidelines remaining informal. It also points out that usable presentations of type theories omit most type annotations, which makes it unclear how to type terms algorithmically; bidirectional typing is the proposed answer.

In Felicissimo's framework a theory is given as data: sorts, constructors, destructors with a distinguished principal argument, and rewrite rules describing how destructors compute on constructors. The Pfenning recipe is then applied uniformly: constructors are checked, destructors are inferred from their principal argument, and a switch rule compares types up to conversion. The key technical device is matching modulo rewriting, in which an expected or inferred type is reduced until its head is a constructor and the pieces can be read off. The paper proves that the bidirectional system is sound with respect to the declarative one and

complete in the sense of annotability, following Dunfield and Krishnaswami, and that typing is decidable for normalizing theories. The framework is implemented in the theory-independent type checker BiTTs [16], which has been used with several theories, among them variants of Martin-Löf type theory.

The two works differ in aim, a contrast already drawn by Tomilovskaia [8, Sec. 2.6]. Felicissimo's is a formal development, with proofs of soundness, annotability, and decidability, in which entire type theories are the objects from which a checker is derived. The present thesis provides no comparable metatheory. Its aim is practical: an engine embedded in a host language, in which the language designer writes typing rules per syntax node against a generic scope-aware traversal over first-order signatures extended with binders. Both approaches lighten the load on language designers, and BiTTs serves as a reference point for the MLTT case study.

2.9 Predecessor Work

The starting point of this thesis is a line of bachelor's theses supervised by Nikolai Kudasov at Innopolis University, and a related workshop paper by Smirnova, Tomilovskaia and Deryabkin [17]. Table 2.1 summarises them.

TABLE 2.1
Predecessor theses on generic type checking over Free Foil

Work	Contribution	Open point
Tomilovskaia [8]	Generic bidirectional engine; System F case study	Rules written by hand; single-variable binders; minimal error localisation
Smirnova [11]	Generic Algorithm W for Hindley–Milner inference	Error localisation; no benchmarks against other checkers
Deryabkin [12]	Hindley–Milner inference with level-based generalisation and separated constraint generation and solving	Hand-written unification; no benchmarks; generic interface only proposed
Meslmani [13]	Generic engine extended to modal typing (IS4 and IK) through an abstract context interface	Non-dependent type language; error-message usability not assessed

Tomilovskaia [8] is the immediate technical predecessor and the starting point for the present thesis. It develops a generic bidirectional engine in Haskell on top of Free Foil. Its central device is a record wrapping each syntax node that exposes both a check function and an infer function, together with a scoped variant that takes an optional type for the binder, so that constructs such as let, where the bound variable's type is known only after inferring the bound expression,

can be handled uniformly. Language designers implement a typeclass with one checking case and one inference case per construct, and the framework handles traversal and scope. The thesis reports a case study of System F in which the generic implementation needs 128 lines of type-checking logic against 199 for a direct Free Foil implementation, a reduction of about 35%, and contributes several utilities to Free Foil, notably an operation for coercing a syntax tree to a smaller scope when it does not use the extra names.

The present thesis is a continuation of this design rather than an independent reimplementation of the same contribution. It takes the generic, scope-aware separation of traversal from language-specific rules as its foundation, and investigates how the interface can be developed into a more general and useful type-elaboration interface. In particular, the result of typing is treated as a complete typed abstract syntax tree, not merely as the type inferred for the root term. The interface and its presentation are also developed with compositional use and practical language examples in mind. This changes the emphasis of the work: the question is not simply how to factor bidirectional checking out of a language implementation, but how to provide a reusable interface through which language-specific typing rules can produce typed syntax.

Tomilovskaia's thesis also records limitations that provide context for further development: rules are written by hand rather than automatically derived from the Pfenning recipe, binders are limited to a single variable, and error localisation is minimal. These are useful points of comparison, but they should not be read as a claim that the present thesis necessarily resolves every limitation. The focus here is the type-elaboration interface and the production of typed syntax.

Smirnova [11] follows the same idea for type inference. The thesis presents a language-agnostic implementation of Algorithm W over second-order abstract

syntax, decoupling the description of the target language from the logic of the algorithm, and verifies it on a lambda calculus with let-expressions and for-expressions. It tests generality by adding pairs to the language, which required only a new grammar rule and new instructions in the user interface and no change to the algorithm. In a comparison of the code a user writes, the instructions for addition and let take 10 lines through the generic interface against 16 without it. The thesis names better error localisation, documentation, and benchmarking against other checkers as next steps.

Deryabkin [12] studies Hindley–Milner-style inference with level-based generalisation, a variant that avoids traversing the whole type environment at let-generalisation, with constraint generation separated from solving, again over Free Foil syntax. The thesis reports that scope safety and generality removed an entire class of bugs and that the type checker is parameterised over the syntax signature, so that new terms do not require changes to the inference core, while noting that the main drawback of Free Foil is its learning curve. Its limitations are a hand-written unification algorithm and the absence of benchmarks, and it proposes a type class, $\mathrm{HMTypingSig}$, as a way towards a generic library. Both inference theses are accompanied by the repository [18].

Meslmani [13] extends the bidirectional engine to a modal setting. The thesis adds Fitch-style box modalities (IS4 and IK) by isolating modal behaviour in an abstract, lock-aware context interface, so that the generic engine commits to no particular modal logic. Its instantiated language agrees with a hand-written baseline checker on all of the baseline's 211 example programs. Its stated limitations are relevant here: the type language is non-dependent, the evidence is example-based rather than metatheoretic, and error-message usability is not assessed.

Taken together, this line of work establishes that a generic, scope-safe bidirec-

tional engine is feasible end to end, from grammar to checker, and that it reduces implementation effort. It also shows the recurring pattern the present thesis builds on: traversal and scope management are factored out, while language-specific rules stay local.

2.10 Related Languages

Two further works serve as comparisons or potential consumers of the engine. Stella [2] is a statically typed language designed for teaching the implementation of type systems. It has a minimalistic core and a set of small, explicitly declared extensions, covering algebraic data types, references, exceptions, exhaustive pattern matching, subtyping, recursive types, universal polymorphism, and type reconstruction, and it relies on the BNF Converter so that templates can be provided in several implementation languages. A language that is organised as a core plus extensions is a natural test of the extensibility that this thesis aims for.

Rzk [9] is a proof assistant for synthetic ∞-categories implementing a variant of Riehl and Shulman's simplicial type theory. It is implemented in Haskell, generates its parser and pretty-printer with the BNF Converter, and uses Free Foil based representation of terms. Its type checker is bidirectional, in the style of Coquand and of Dunfield and Krishnaswami, and extends the standard picture so that every judgement carries an additional tope context. Type checking runs over an annotated copy of the syntax in which every node carries its checked type together with cached normal forms. This illustrates the usefulness of retaining typing information throughout a syntax tree, rather than exposing only the type of the root term. It is therefore a relevant point of comparison for the typed output considered in this thesis. Because the checker is specific to the theory it

implements, Rzk is also a realistic example of the kind of dependent type checker that a generic engine could serve.

2.11 Conclusion

This chapter surveyed the foundations and prior work behind a reusable type-elaboration interface. It recalled the simply typed λ-calculus and System F, which Tomilovskaia treats in detail, and introduced Hindley–Milner inference, whose need for a unification constraint context sets it apart from them. It then covered bidirectional typing as a discipline for implementing typing rules, scope-safe abstract syntax with a focus on Free Foil, the Pfenning recipe, and other approaches to generic type checking.

The predecessor work establishes that generic, scope-safe type checking is a practical approach: common machinery can be separated from language-specific typing rules, and the resulting framework can be applied to more than one language. Tomilovskaia's engine is the immediate foundation of this thesis. The present work builds on that foundation but shifts the emphasis towards the interface for type elaboration and the form of its output. In particular, elaboration means returning a complete typed abstract syntax tree, with typing information associated with the syntax nodes, rather than returning only the type of the current or root expression.

The contribution of this thesis is to develop a reusable interface for defining language-specific checking and inference rules and producing typed syntax, together with further examples that demonstrate its use. The intended separation is that the language author supplies the grammar and the rules for the language's syntax constructs, while the framework provides the shared machinery needed to

apply those rules over scope-safe syntax and construct the elaborated result. The approach is evaluated through concrete language implementations, including System F and Martin-Löf type theory. These examples provide a basis for assessing the practicality of the interface, the reuse of its common machinery, and its ability to produce complete typed syntax.

A direct extension of Tomilovskaia's work is a legitimate research contribution. The distinction lies in the question being addressed: rather than focusing only on how to implement generic bidirectional type checking, this thesis investigates how to provide a reusable and practical interface for type elaboration over scope-safe syntax.

Chapter 3

Methodology

Referencing other chapters 2, 3, 4, 5 and 6

TABLE 3.1
Simulation Parameters

A	B
Parameter	**Value**
Number of vehicles	$\mid \mathcal{V} \mid$
Number of RSUs	$\mid \mathcal{U} \mid$
RSU coverage radius	150 m
V2V communication radius	30 m
Smart vehicle antenna height	1.5 m
RSU antenna height	25 m
Smart vehicle maximum speed	v_{max} m/s
Smart vehicle minimum speed	v_{min} m/s
Common smart vehicle cache capacities	$[50, 100, 150, 200, 250]$ mb
Common RSU cache capacities	$[5000, 1000, 1500, 2000, 2500]$ mb

A	B
Common backhaul rates	$[75, 100, 150]$ mb/s

...

Chapter 4

Implementation

...

Chapter 5

Evaluation and Discussion

...

Chapter 6

Conclusion

...

Bibliography cited

[1] T. Felicissimo, *Generic bidirectional typing for dependent type theories*. arXiv: 2307.08523.

[2] A. Abounegm, N. Kudasov, and A. Stepanov, "Teaching type systems implementation with Stella, an extensible statically typed programming language," *Electronic Proceedings in Theoretical Computer Science*, vol. 405, pp. 1–19, 2024. DOI: 10.4204/EPTCS.405.1 arXiv: 2407.08089.

[3] J. Dunfield and N. Krishnaswami, "Bidirectional typing," *ACM Computing Surveys*, vol. 54, no. 5, 2021. DOI: 10.1145/3450952 arXiv: 1908.05839.

[4] D. Maclaurin, A. Radul, and A. Paszke, "The foil: Capture-avoiding substitution with no sharp edges," in *Proceedings of the 34th Symposium on Implementation and Application of Functional Languages (IFL '22)*, 2023. DOI: 10.1145/3587216.3587224

[5] G. Allais, R. Atkey, J. Chapman, C. McBride, and J. McKinna, "A type- and scope-safe universe of syntaxes with binding: Their semantics and proofs," *Journal of Functional Programming*, vol. 31, e22, 2021. DOI: 10.1017/S0956796820000076

[6] N. Kudasov, R. Shakirova, E. Shalagin, and K. Tyulebaeva, "Free foil: Generating efficient and scope-safe abstract syntax," in *2024 4th International Conference on Code Quality (ICCQ)*, 2024, pp. 1–16. arXiv: 2405.16384.

[7] *Free foil library*, https://github.com/fizruk/free-foil.

[8] D. Tomilovskaia, "Bidirectional typechecking via second-order abstract syntax," Bachelor's thesis, Innopolis University, 2025. [Online]. Available: https://e.lanbook.com/vkr/201994

[9] N. Kudasov, V. Sim, and B. Ahrens, *Rzk: A proof assistant for synthetic ∞-categories*, 2026. arXiv: 2607.12207.

[10] N. Kudasov, "Free monads, intrinsic scoping, and higher-order preunification," in *Trends in Functional Programming (TFP 2024)*, 2024, pp. 22–54. arXiv: 2204.05653.

[11] A. Smirnova, "Generalization of Hindley-Milner type inference using Second-Order Abstract Syntax," Bachelor's thesis, Innopolis University, 2025.

[12] V. Deryabkin, "Hindley-milner-style type inference with levels for generic abstract syntax with binders," Bachelor's thesis, Innopolis University, 2025. [Online]. Available: https://e.lanbook.com/vkr/202027

[13] A. Meslmani, "Context locks and bidirectional modal typing for abstract syntax with binders," Bachelor's thesis, Innopolis University, 2026.

[14] W. Swierstra, "Data types à la carte," *Journal of Functional Programming*, vol. 18, no. 4, pp. 423–436, 2008. DOI: 10.1017/S0956796808006758

[15] S. Peyton Jones and S. Marlow, "Secrets of the Glasgow Haskell Compiler inliner," *Journal of Functional Programming*, vol. 12, pp. 393–434, 2002.

[16] *BiTTs: Generic bidirectional type checker*, https://github.com/thiagofelicissimo/BiTTs.

[17] N. Kudasov, A. Smirnova, V. Deryabkin, D. Tomilovskaia, and E. Maksimova, "Towards generic type checking implementations in Haskell via second-order abstract syntax," in *Workshop on the Implementation of Type Systems (WITS)*, 2025.

[18] *Free-foil-typecheck*, https://github.com/evermake/free-foil-typecheck.

Appendix A

Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Appendix B

Even More Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.